BANCO DE DADOS AVANÇADO

Transações no MySQL

Professor: Claudinei José Machado

Fonte: Inter

MySQL 9 • Banco de dados: ecommerce

1. Objetivos da aula

- Compreender o conceito de transação e sua importância para consistência e confiabilidade.
- Utilizar START TRANSACTION, COMMIT, ROLLBACK e SAVEPOINT.
- Aplicar transações em operações críticas.
- Simular processamento de pedidos e transferências financeiras.
- Identificar situações em que uma operação pode deixar dados parcialmente processados.

2. O que é uma transação?

Transação é um conjunto de operações tratado como uma única unidade lógica de trabalho.

Ideia central: todas as operações são confirmadas ou nenhuma delas deve produzir efeito permanente.

Exemplo: uma compra pode envolver criação do pedido, inclusão dos itens, atualização do estoque e atualização do status.

3. Por que transações são importantes?

Operações críticas normalmente alteram mais de um dado ou mais de uma tabela.

- Pagamentos
- Transferências
- Compras
- Reservas
- Controle de estoque
- Processamento de pedidos

Problema: se apenas parte da operação for executada, o banco pode ficar inconsistente.

4. Exemplo: operação sem transação

```
INSERT INTO orders (...);

INSERT INTO order_items (...);

UPDATE products
SET stock = stock - 2
WHERE id = 10;
```

Se o terceiro comando falhar, podemos ter pedido e item registrados, mas estoque não atualizado.

5. Exemplo: operação com transação

```
START TRANSACTION;

INSERT INTO orders (...);

INSERT INTO order_items (...);

UPDATE products
SET stock = stock - 2
WHERE id = 10;

COMMIT;
```

Se houver uma falha antes da confirmação, podemos desfazer a unidade de trabalho com ROLLBACK.

6. Atomicidade: tudo ou nada

Atomicidade significa tratar a transação como uma unidade indivisível.

Transferência de R$ 200:

Conta A → − R$ 200
Conta B → + R$ 200

Não queremos uma situação em que apenas o débito seja realizado.

7. ACID

Propriedade	Significado
Atomicidade	A transação é tratada como uma unidade: tudo ou nada.
Consistência	O banco deve permanecer em estados consistentes.
Isolamento	Transações concorrentes não devem expor estados intermediários de forma inadequada.
Durabilidade	Após COMMIT, as alterações devem permanecer persistidas dentro das garantias do SGBD.

8. InnoDB no banco ecommerce

O schema utilizado na aula informa tabelas do banco ecommerce usando o mecanismo InnoDB.

```
CREATE TABLE products (
id BIGINT UNSIGNED NOT NULL,
name VARCHAR(255) NOT NULL,
stock BIGINT NOT NULL,
price DECIMAL(10,2) NOT NULL
) ENGINE=InnoDB;
```

As tabelas products, orders e order_items são adequadas para os exemplos transacionais desta aula.

9. Autocommit

O MySQL normalmente trabalha com autocommit habilitado.

```
SELECT @@autocommit;
```

Quando o resultado é 1, alterações individuais podem ser confirmadas automaticamente.

Para agrupar várias operações em uma unidade de trabalho, iniciamos explicitamente uma transação.

10. START TRANSACTION

```
START TRANSACTION;

UPDATE products
SET stock = stock - 1
WHERE id = 1;

UPDATE products
SET stock = stock - 2
WHERE id = 2;

COMMIT;
```

Inicia uma transação explícita. BEGIN também pode ser usado como alternativa.

11. COMMIT

COMMIT confirma as alterações realizadas na transação.

```
START TRANSACTION;

UPDATE products
SET stock = stock - 1
WHERE id = 1;

COMMIT;
```

Depois do COMMIT, um ROLLBACK posterior não desfaz essa alteração já confirmada.

12. ROLLBACK

ROLLBACK desfaz as alterações realizadas desde o início da transação, enquanto elas ainda não foram confirmadas.

```
START TRANSACTION;

UPDATE products
SET stock = stock - 5
WHERE id = 1;

ROLLBACK;
```

13. Experimento com ROLLBACK

```
SELECT id, name, stock
FROM products
WHERE id = 1;

START TRANSACTION;

UPDATE products
SET stock = stock - 10
WHERE id = 1;

SELECT id, name, stock
FROM products
WHERE id = 1;

ROLLBACK;

SELECT id, name, stock
FROM products
WHERE id = 1;
```

Compare os valores antes, durante e depois do ROLLBACK.

14. SAVEPOINT

Um SAVEPOINT cria um ponto intermediário dentro da transação.

```
START TRANSACTION;

UPDATE products
SET stock = stock - 1
WHERE id = 1;

SAVEPOINT estoque_atualizado;

UPDATE products
SET stock = stock - 2
WHERE id = 2;
```

Podemos voltar ao ponto salvo sem necessariamente desfazer toda a transação.

15. ROLLBACK TO SAVEPOINT

```
ROLLBACK TO SAVEPOINT estoque_atualizado;
```

A operação realizada depois do savepoint é desfeita; a transação continua aberta.

```
COMMIT;
```

Ou, se necessário, podemos desfazer toda a transação com ROLLBACK.

16. RELEASE SAVEPOINT

```
RELEASE SAVEPOINT estoque_atualizado;
```

Remove o ponto de recuperação.

Importante: RELEASE SAVEPOINT não confirma a transação.

17. Comandos principais

Comando	Função
START TRANSACTION	Inicia uma transação.
COMMIT	Confirma as alterações.
ROLLBACK	Desfaz as alterações da transação.
SAVEPOINT nome	Cria um ponto intermediário.
ROLLBACK TO SAVEPOINT nome	Retorna a um ponto intermediário.
RELEASE SAVEPOINT nome	Remove um savepoint.

18. Estrutura do ecommerce

Para os exemplos, as principais relações são:

customers → orders → order_items → products

O schema possui chaves estrangeiras entre essas tabelas, incluindo ON DELETE CASCADE em order_items para order_id e product_id.

19. Processamento de pedido

Uma compra pode envolver:

- Criar o pedido.
- Adicionar os itens.
- Verificar e atualizar o estoque.
- Calcular o total.
- Atualizar o status.

Essas operações podem precisar ser tratadas como uma única unidade lógica.

20. Verificando o produto

```
SELECT id, name, price, stock
FROM products
WHERE id = 1;
```

Antes de alterar o estoque, conheça o estado atual do produto.

21. Atualização segura do estoque

```
UPDATE products
SET stock = stock - 2,
updated_at = NOW()
WHERE id = 1
AND stock >= 2;

SELECT ROW_COUNT();
```

ROW_COUNT() permite verificar se a atualização ocorreu. Resultado 0 indica que a condição não foi atendida ou o registro não foi encontrado.

22. Pedido: transação completa

```
START TRANSACTION;

INSERT INTO orders (
customer_id, total, status,
created_at, updated_at
)
VALUES (1, 0.00, 'open', NOW(), NOW());

SET @order_id = LAST_INSERT_ID();

INSERT INTO order_items (
order_id, product_id,
product_name, product_price,
quantity, created_at, updated_at
)
SELECT
@order_id, id, name, price,
2, NOW(), NOW()
FROM products
WHERE id = 1;
```

23. Pedido: estoque e total

```
UPDATE products
SET stock = stock - 2,
updated_at = NOW()
WHERE id = 1
AND stock >= 2;

UPDATE orders
SET total = (
SELECT SUM(product_price * quantity)
FROM order_items
WHERE order_id = @order_id
)
WHERE id = @order_id;

COMMIT;
```

Em uma aplicação real, as validações e decisões de erro devem determinar se o fluxo termina em COMMIT ou ROLLBACK.

24. Transferência financeira

O banco ecommerce não possui tabela de contas bancárias. Para este exemplo didático, podemos criar uma tabela hipotética.

```
CREATE TABLE accounts (
id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
holder_name VARCHAR(255) NOT NULL,
balance DECIMAL(10,2) NOT NULL DEFAULT 0.00
) ENGINE=InnoDB;
```

25. Transferência com transação

```
START TRANSACTION;

UPDATE accounts
SET balance = balance - 200
WHERE id = 1
AND balance >= 200;

UPDATE accounts
SET balance = balance + 200
WHERE id = 2;

COMMIT;
```

Se houver problema durante o processamento, a operação deve ser revertida antes da confirmação.

26. Validação antes do COMMIT

```
START TRANSACTION;

UPDATE accounts
SET balance = balance - 200
WHERE id = 1
AND balance >= 200;

UPDATE accounts
SET balance = balance + 200
WHERE id = 2;

SELECT id, holder_name, balance
FROM accounts
WHERE id IN (1, 2);
```

Depois da validação: COMMIT para confirmar ou ROLLBACK para desfazer.

27. SAVEPOINT em operação complexa

```
START TRANSACTION;

-- Criar pedido
INSERT INTO orders (...);

SAVEPOINT pedido_criado;

-- Adicionar itens
INSERT INTO order_items (...);

-- Atualizar estoque
UPDATE products
SET stock = stock - 1
WHERE id = 1;

SAVEPOINT estoque_atualizado;
```

Um savepoint permite criar pontos intermediários de recuperação.

28. Transação × SAVEPOINT

Transação	SAVEPOINT
Agrupa uma unidade de trabalho.	Cria um ponto intermediário.
Termina normalmente com COMMIT ou ROLLBACK.	Pode ser usado com ROLLBACK TO SAVEPOINT.
Controla a confirmação/desfazer da unidade.	Não confirma a transação.

29. phpMyAdmin

Os comandos podem ser executados diretamente na aba SQL do phpMyAdmin.

```
START TRANSACTION;

UPDATE products
SET stock = stock - 1
WHERE id = 1;

SELECT id, name, stock
FROM products
WHERE id = 1;

ROLLBACK;
```

Depois, consulte novamente o estoque para observar o efeito.

30. Terminal MySQL

```
mysql -u root -p ecommerce
```

```
START TRANSACTION;

SELECT id, name, stock
FROM products
WHERE id = 1;

UPDATE products
SET stock = stock - 1
WHERE id = 1;

ROLLBACK;
```

O mesmo controle transacional pode ser realizado pelo cliente de linha de comando.

31. Concorrência

Transações também estão relacionadas ao comportamento de múltiplas conexões trabalhando simultaneamente.

Modelo conceitual:

Sessão A → Transação A
Sessão B → Transação B

Questões como bloqueios, níveis de isolamento, SELECT ... FOR UPDATE e deadlocks fazem parte do estudo mais aprofundado de concorrência.

32. O que uma transação não resolve sozinha?

START TRANSACTION não substitui regras de negócio, validações e controle de concorrência.

```
SELECT stock
FROM products
WHERE id = 1;

UPDATE products
SET stock = stock - 1
WHERE id = 1;
```

Em cenários concorrentes, a forma como os registros são lidos e alterados pode exigir mecanismos adicionais.

33. Atenção com DDL

Comandos como CREATE TABLE, ALTER TABLE e DROP TABLE são operações de definição de estrutura.

No MySQL, determinados comandos DDL podem provocar implicit commit. Portanto, não devemos assumir que qualquer comando SQL poderá ser desfeito por ROLLBACK.

34. Boas práticas

- Mantenha transações objetivas e curtas.
- Defina claramente quais operações pertencem à unidade de trabalho.
- Valide condições críticas antes do COMMIT.
- Não confirme uma etapa antes de concluir as demais etapas que precisam ser atômicas.
- Use SAVEPOINT quando pontos intermediários de recuperação forem realmente necessários.

35. Modelo mental

1. Quais tabelas serão alteradas?
2. Quais operações precisam acontecer juntas?
3. O que acontece se uma operação falhar?
4. Em que momento tudo está correto? → COMMIT
5. O que fazer se houver erro? → ROLLBACK
6. Existe uma etapa intermediária recuperável? → SAVEPOINT

36. Fluxo completo

START TRANSACTION

↓

Operação 1 → Operação 2 → SAVEPOINT → Operação 3

↓

Sucesso → COMMIT

Erro parcial → ROLLBACK TO SAVEPOINT

Erro geral → ROLLBACK

37. Atividade 1 — ROLLBACK

Utilizando products:

- Selecione um produto e anote o estoque.
- Inicie uma transação.
- Reduza o estoque em 3 unidades.
- Consulte o estoque.
- Execute ROLLBACK.
- Consulte novamente.

Objetivo: comprovar o funcionamento do ROLLBACK.

38. Atividade 2 — COMMIT

Utilizando products:

- Escolha um produto.
- Inicie uma transação.
- Altere o preço.
- Consulte o novo valor.
- Execute COMMIT.
- Tente executar ROLLBACK e consulte novamente.

Objetivo: compreender a confirmação definitiva da transação.

39. Atividade 3 — SAVEPOINT

Faça três alterações em products.

Crie um SAVEPOINT depois da segunda alteração.

Faça uma terceira alteração.

Execute ROLLBACK TO SAVEPOINT.

Verifique quais alterações permaneceram.

Finalize com COMMIT.

40. Atividade 4 — Processamento de pedido

Utilizando customers, orders, order_items e products:

- Inicie uma transação.
- Crie um pedido.
- Insira pelo menos dois itens.
- Verifique e atualize o estoque.
- Calcule o total.
- Valide os dados.
- Finalize com COMMIT ou ROLLBACK.

Desafio: não permita estoque negativo.

41. Atividade 5 — Pedido com falha

Provoque uma situação em que uma etapa posterior do processamento apresente erro.

A solução deve impedir que pedido, itens e estoque permaneçam parcialmente processados.

Objetivo: demonstrar por que operações críticas devem ser tratadas como uma unidade.

42. Atividade 6 — Sistema bancário

Crie accounts com id, holder_name e balance.

- Insira pelo menos três contas.
- Implemente uma transferência.
- Valide o saldo da origem.
- Use START TRANSACTION, COMMIT e ROLLBACK.
- Não permita saldo negativo.

43. Atividade 7 — Transferência com SAVEPOINT

Monte uma transferência com:

Débito → SAVEPOINT → Crédito → outra operação

Provoque um erro depois do SAVEPOINT.

Use ROLLBACK TO SAVEPOINT e decida se a transação poderá continuar ou deverá ser totalmente desfeita.

44. Atividade 8 — Análise de atomicidade

Considere:

1. Criar pedido
2. Inserir item
3. Reduzir estoque
4. Registrar pagamento
5. Atualizar status

- Quais operações devem fazer parte da mesma transação?
- O que acontece se o banco falhar entre as etapas 2 e 3?
- O que aconteceria se houvesse COMMIT após a etapa 3?
- Em que situação usar ROLLBACK?
- Em que situação um SAVEPOINT poderia ser útil?

45. Desafio final

Desenvolva uma operação completa de compra utilizando o banco ecommerce.

Fluxo: cliente → pedido → produtos → estoque → total → status → COMMIT

Caso qualquer etapa crítica apresente problema: ROLLBACK.

A solução deve utilizar obrigatoriamente START TRANSACTION, COMMIT, ROLLBACK e pelo menos um SAVEPOINT.

46. Entrega do desafio

- Código SQL.
- Consultas utilizadas para validação.
- Evidências do funcionamento com COMMIT.
- Evidências do funcionamento com ROLLBACK.
- Explicação de onde o SAVEPOINT foi utilizado.
- Explicação de como a solução evita inconsistência.

47. Síntese

Transações permitem tratar operações relacionadas como uma unidade de trabalho.

START TRANSACTION → inicia

COMMIT → confirma

ROLLBACK → desfaz

SAVEPOINT → cria ponto intermediário

O objetivo não é apenas conhecer os comandos, mas saber quando e por que utilizá-los para proteger operações de negócio.